VIDEO MONITORING SYSTEM

GStreamer Assignment

Philippe FOUBERT

8. November 2026

Philippe FOUBERT
GStreamer

- Introduction
- Part 1: Pipeline architecture and command-line implementation
- Part 2: Architecture documentation
- Part 3: Native sender application
- Part 4: Debugging report
- Part 5: Engineering experiment
- Part 6: Automated verification
- Details

Philippe FOUBERT
GStreamer

INTRODUCTION

Philippe FOUBERT
GStreamer

PURPOSE

- In this assignment, you will design, implement, test, and diagnose a video monitoring system using GStreamer.
- The system will:
  - generate a live video stream;
  - display the stream locally;
  - record the stream to a file;
  - transmit the stream over RTP/UDP;
  - receive and display the network stream;
  - implement the sender as a native C or C++ application;
  - provide an automated verification procedure.
- You will complete the work in the provided GitHub Codespaces environment. No camera or physical audio device is required.

Philippe FOUBERT
GStreamer

LEARNING OBJECTIVES

By completing this assignment, you should be able to:

- construct non-trivial GStreamer pipelines;
- explain the roles of elements, pads, caps, encoders, payloaders, muxers, and sinks;
- use tee and queue to implement independent processing branches;
- transmit and receive an RTP stream;
- create a playable recording and finalize it correctly;
- control a pipeline from a native application;
- process messages received from the GStreamer bus;
- diagnose caps, plugin, transport, and pipeline-state problems;
- measure selected performance characteristics;
- create reproducible technical tests;
- use AI tools critically and validate their suggestions;
- collaborate effectively on a technical implementation.

Philippe FOUBERT
GStreamer

WORKING CONDITIONS

- **Team size**: One to three students
- **Environment**: Supplied GitHub repository and GitHub Codespaces environment
- **Implementation language**: C or C++
- **Submission**: One Git repository per team
- **Assessment**: Repository contents and automated or reproducible verification by the teacher
- **AI tools**: Permitted, subject to the disclosure requirements in Section 11
- **Mark**: Out of 20

Philippe FOUBERT
GStreamer

- A student may complete the assignment individually. Teams may contain a maximum of three students.
- All team members are responsible for the submitted solution. Dividing the implementation work does not remove the requirement to review and understand the complete system.
- There will be no scheduled live demonstration or individual oral examination. The teacher must therefore be able to build, run, test, and evaluate the submission independently by following the repository instructions.

Philippe FOUBERT
GStreamer

TEAM CONFIGURATION

Each team will receive a configuration containing some or all of the following values:

Parameter	Possible values
Video pattern	smpte, ball, snow, or zone-plate
Resolution	640×360, 800×450, or 960×540
Frame rate	5, 10, or 15 fps
RTP/UDP port	A value between 5000 and 5999
Overlay position	Top-left, top-center, or bottom-right
Recording container	Matroska or MPEG-TS
Network codec	H.264 or JPEG
Required runtime	45, 60, or 90 seconds

You must use the values assigned to your team.

Record the assigned values in a file named: `config/team.conf`

Philippe FOUBERT
GStreamer

Use a simple format that can be read by a shell script:

```
PATTERN=smpte
WIDTH=800
HEIGHT=450
FPS=10
PORT=5123
OVERLAY_POSITION=top-center
CONTAINER=matroska
NETWORK_CODEC=h264
RUNTIME=60
```

The values above are examples. Use the configuration assigned to your team.

Philippe FOUBERT
GStreamer

PART 1: PIPELINE ARCHITECTURE AND COMMAND-LINE IMPLEMENTATION

Philippe FOUBERT
GStreamer

SOURCE PROCESSING

- Create a live video stream using videotestsrc.
- The source pipeline must:
    1. use your assigned test pattern;
    2. produce your assigned resolution;
    3. produce your assigned frame rate;
    4. overlay the names or identifiers of all team members;
    5. overlay a running timestamp or elapsed time;
    6. display the configured resolution and frame rate.
- After this common processing, use tee to divide the stream into three branches.

Philippe FOUBERT GStreamer

BRANCH A: LOCAL PREVIEW

- The first branch must display the stream in the virtual desktop.
- Requirements:
  - Use an appropriate video sink.
  - Place a queue after the tee.
  - Ensure that activity on another branch does not unnecessarily block the preview.
  - Explain the purpose of the queue in architecture.md.
- The verification procedure must also support a non-interactive mode in which the display sink is replaced by an appropriate sink such as fakesink. This allows the teacher to test the pipeline even when a graphical display is unavailable.

Philippe FOUBERT
GStreamer

BRANCH B: RECORDING

- The second branch must record the video to a file.
- Requirements:
  - Use the assigned recording container.
  - Select an appropriate video encoder.
  - Use an appropriate parser or muxer where required.
  - Finalize the file correctly by sending EOS before terminating the pipeline.
  - Verify that the resulting file is playable.
  - Provide a separate GStreamer command that plays the recording.
- You must not use kill -9 or equivalent forced termination as the normal way to finish the recording.
- The automated verification must produce a recording in:
  `artifacts/test-recording.<extension>`
  The file extension must match the selected container.

Philippe FOUBERT GStreamer

BRANCH C: RTP/UDP TRANSMISSION

- The third branch must transmit the live stream over RTP/UDP.

- Requirements:

  - Use your assigned codec.
  - Encode the video.
  - Packetize the encoded stream using the appropriate RTP payloader.
  - Send the resulting packets to localhost on your assigned port.
  - Provide a separate receiver pipeline.
  - Specify receiver-side RTP caps where necessary.

- The receiver must support two modes:

  1. **display mode**, which displays the received video;
  2. **test mode**, which decodes the received video and uses a non-display sink.

  Test mode must make it possible to verify reception without manually observing a window.

Philippe FOUBERT GStreamer

Your documentation must explain:

- the difference between an encoder and an RTP payloader;
- why the receiver needs an RTP depayloader;
- why the receiver may need explicit caps;
- which parts of the pipeline use raw video;
- which parts use compressed video;
- the purpose of the parser, where one is used;
- the purpose of the recording muxer.

Philippe FOUBERT
GStreamer

REQUIRED SCRIPTS

Provide the following executable scripts:

- scripts/sender.sh
- scripts/receiver.sh
- scripts/play-recording.sh
- scripts/build.sh
- scripts/verify.sh
- scripts/clean.sh

Philippe FOUBERT
GStreamer

Each script must:

- start with an appropriate shebang;
- return zero when successful;
- return a non-zero exit status when an error occurs;
- use clearly named variables;
- print meaningful progress and error messages;
- work when launched from the root of the repository;
- not require the teacher to edit the script.

Philippe FOUBERT
GStreamer

PART 2: ARCHITECTURE DOCUMENTATION

Philippe FOUBERT
GStreamer

CREATE A FILE NAMED ARCHITECTURE.MD.

- It must include:

    1. a diagram of the complete sender;
    2. the names and purposes of the main GStreamer elements;
    3. the raw-video caps used before the tee;
    4. the codec, RTP payload, and UDP port used for transmission;
    5. the codec and container used for recording;
    6. an explanation of each queue;
    7. an explanation of how the receiver reconstructs the stream;
    8. a description of how the recording is finalized;
    9. a description of the differences between normal mode and test mode.

- You may draw the architecture using Mermaid, PlantUML, ASCII, or another readable diagram format.

- Your diagram must make the three processing branches clearly visible.

Philippe FOUBERT
GStreamer

For example:

```
Video source
    |
Video conversion and caps
    |
Overlay
    |
   tee
  / | \
 /  |  \
Preview  Recording  RTP/UDP transmission
```

This example shows only the expected general structure. It is not a complete solution.

Philippe FOUBERT
GStreamer

PIPELINE GRAPH

- Generate at least one GStreamer DOT graph using:

```
GST_DEBUG_DUMP_DOT_DIR=...
```

- Convert the graph to PNG and save it as: `evidence/pipeline.png`
- The graph must come from your implemented pipeline.

Philippe FOUBERT GStreamer

PART 3: NATIVE SENDER APPLICATION

Philippe FOUBERT
GStreamer

GENERAL REQUIREMENT

- Replace the shell-based sender with a native executable named: `monitor_sender`
- You may implement it in C or C++.
- The receiver and recording playback tools may remain shell scripts.

Philippe FOUBERT
GStreamer

COMMAND-LINE INTERFACE

Your executable must accept, at minimum:

```
monitor_sender \
  --pattern smpte \
  --width 800 \
  --height 450 \
  --fps 10 \
  --port 5123 \
  --output recording.mkv
```

The values above are examples. Your program must work with the configuration assigned to your team.

Philippe FOUBERT
GStreamer

It must:

- display a useful –help message;
- reject missing required arguments;
- reject invalid numeric values;
- reject unsupported patterns or file formats;
- print meaningful error messages;
- return a non-zero exit status after a fatal error.

Hard-coding all parameters in the source code is not acceptable.

Philippe FOUBERT
GStreamer

PIPELINE IMPLEMENTATION

You may choose one of the following approaches.

- **Option A: Manual construction**

  Create and connect the GStreamer elements programmatically, using functions such as:

  ```
  gst_element_factory_make()
  ```

- **Option B: Parsed pipeline**

  Create the main pipeline using:

  ```
  gst_parse_launch()
  ```

  Use element names to retrieve elements that must be controlled while the pipeline is running.

Explain your choice in architecture.md, including at least one advantage and one limitation.

Philippe FOUBERT GStreamer

RUNTIME BEHAVIOR

- The application must:

    1. initialize GStreamer;
    2. create the sender pipeline;
    3. set the pipeline to PLAYING;
    4. run a GLib main loop;
    5. process messages received from the GStreamer bus;
    6. detect and report pipeline errors;
    7. report the element that produced an error;
    8. print the available GStreamer debug information;
    9. handle EOS;
    10. stop the pipeline by setting it to NULL;
    11. release its GStreamer and GLib resources.

Philippe FOUBERT GStreamer

- The application must support clean, unattended termination after a specified duration.
  Add the following option:

```
--duration <seconds>
```

  For example:

```
./build/monitor_sender \
--pattern smpte \
--width 800 \
--height 450 \
--fps 10 \
--port 5123 \
--output artifacts/test-recording.mkv \
--duration 15
```

- When the duration expires, the application must send EOS, finalize the recording, stop the pipeline, release its resources, and return zero.

- This mechanism is mandatory because it allows the teacher to run the submission without manually stopping the process.

Philippe FOUBERT
GStreamer

HEADLESS TEST MODE

- Add the following option:

```
--headless
```

  In headless mode, the preview branch must use a non-display sink while the recording and network branches remain operational.

  The combination of these options must work:

```
--duration 15 --headless
```

Philippe FOUBERT
GStreamer

DYNAMIC PROPERTY

- While the pipeline is running, change at least one visible property every five seconds.
- Possible properties include:
  overlay text, brightness, contrast, saturation, test-pattern type.
- The change must happen without rebuilding the complete pipeline.
- Document the selected property and corresponding named element in README.md.
- In headless mode, the application must log every dynamic property change so that the behavior can be verified without looking at the video.
- Example:

```
DYNAMIC_PROPERTY element=video_balance property=brightness value=0.20
```

Philippe FOUBERT GStreamer

PART 4: DEBUGGING REPORT

Investigate and document one significant problem encountered while developing the assignment.

Philippe FOUBERT
GStreamer

The problem may concern:

- incompatible caps;
- missing queue;
- incorrect RTP payloader or depayloader;
- incorrect RTP payload type;
- incomplete receiver caps;
- port mismatch;
- unavailable element;
- incorrect GST_PLUGIN_PATH;
- incorrect container or muxer;
- recording terminated without EOS;
- pipeline state changes;
- resource cleanup.

Use at least two of the following diagnostic mechanisms:

```
gst-inspect-1.0
gst-launch-1.0 -v
GST_DEBUG=2
GST_DEBUG="*:2,GST_CAPS:5"
GST_DEBUG_DUMP_DOT_DIR=...
```

You may use additional commands and GStreamer debug categories.

Philippe FOUBERT
GStreamer

DEBUG REPORT

- Create a file named debug-report.md.

- It must contain:

    1. the observed symptom;
    2. the initial hypotheses;
    3. the commands used during the investigation;
    4. a relevant log extract of no more than 20 lines;
    5. the identified root cause;
    6. the correction;
    7. an explanation of why the correction works;
    8. one alternative diagnosis that was considered and rejected;
    9. the evidence used to reject it;
    10. instructions allowing the teacher to reproduce the original fault, where reasonably possible.

- Do not include hundreds of unfiltered log lines.
  Select the evidence that supports your conclusion.

Philippe FOUBERT
GStreamer

PART 5: ENGINEERING EXPERIMENT

Philippe FOUBERT
GStreamer

PERFORM A CONTROLLED COMPARISON OF TWO CONFIGURATIONS.

- Change only one principal variable.
- Possible comparisons include:
  - with and without a queue on one branch;
  - JPEG/RTP versus H.264/RTP;
  - 5 fps versus 15 fps;
  - 640×360 versus 960×540;
  - two encoder bitrate settings;
  - short GOP versus longer GOP;
  - two queue-size configurations.
- Run each configuration for at least 45 seconds.

Philippe FOUBERT
GStreamer

REQUIRED MEASUREMENTS

- Report the measurements relevant to your comparison. These can include:
  - average CPU usage;
  - output file size;
  - approximate network data rate;
  - startup time;
  - warnings or errors;
  - dropped-buffer observations;
  - measured latency.
- Possible tools include:

```
pidstat -p <pid> 1
du -b <recording>
tcpdump -ni lo -nn 'udp port <port>'
gst-launch-1.0 -v ...
```

- If you measure latency, describe the measurement method. Do not report an unexplained number.

Philippe FOUBERT
GStreamer

EXPERIMENT REPORT

- Create a file named experiment.md.
- It must contain:
  - the question being investigated;
  - the hypothesis;
  - the two configurations;
  - the controlled variables;
  - the measurement method;
  - the results;
  - limitations of the measurement;
  - the engineering conclusion.
- A statement such as “H.264 is better” is not an acceptable conclusion.

Philippe FOUBERT
GStreamer

- The conclusion must relate the result to the video monitoring scenario and consider relevant trade-offs such as:
  - image quality;
  - bitrate;
  - latency;
  - CPU consumption;
  - error recovery;
  - interoperability;
  - implementation complexity.
- Save numerical results in: `evidence/measurements.csv`
- Provide a script named: `scripts/run-experiment.sh`
- The script must reproduce the main experiment or print exact commands for the teacher to run.

Philippe FOUBERT
GStreamer

PART 6: AUTOMATED VERIFICATION

Philippe FOUBERT
GStreamer

VERIFICATION OBJECTIVE

- The teacher must be able to evaluate the functional parts of the submission without editing the code and without manually controlling several terminals.
- Provide:

```
scripts/verify.sh
```

- The teacher will run:

```
./scripts/clean.sh
./scripts/build.sh
./scripts/verify.sh
```

- These commands must be sufficient to perform the main functional verification.

Philippe FOUBERT
GStreamer

REQUIRED VERIFICATION STEPS

- verify.sh must:

1. check that required commands and GStreamer elements are available;
2. check that config/team.conf exists and can be read;
3. check that required source and documentation files exist;
4. verify that monitor_sender –help succeeds;
5. verify that invalid arguments are rejected;
6. start the receiver in test mode;
7. start the sender in headless mode;
8. run the sender for a finite duration;
9. verify that network data was received;
10. verify that the recording file was created;
11. verify that the recording is non-empty;
12. verify that the recording can be discovered or decoded;
13. verify that the sender terminated cleanly;
14. verify that EOS was processed;
15. save the relevant logs;
16. print a final summary.

Philippe FOUBERT GStreamer

- To keep evaluation time reasonable, verify.sh may use a shorter duration than the assigned normal runtime. A verification duration between 10 and 20 seconds is recommended.

Philippe FOUBERT
GStreamer

TIMEOUTS

- No verification step may wait indefinitely.
- Use an appropriate timeout mechanism, for example:

```
timeout 30s <command>
```

- If a timeout occurs, the script must:
  - stop the processes that it started;
  - print an explicit error;
  - return a non-zero exit status.
- The script must not leave sender or receiver processes running after it finishes.

Philippe FOUBERT GStreamer

VERIFICATION OUTPUTS

Create:

```
artifacts/
├── build.log
├── sender.log
├── receiver.log
├── verification.log
├── verification-summary.txt
└── test-recording.<extension>
```

The summary must use a clear format:

```
BUILD=PASS
HELP_OPTION=PASS
INVALID_ARGUMENTS=PASS
SENDER_START=PASS
RECEIVER_START=PASS
RTP_RECEPTION=PASS
EOS_HANDLING=PASS
RECORDING_CREATED=PASS
RECORDING_DECODABLE=PASS
CLEAN_TERMINATION=PASS
OVERALL=PASS
```

Philippe FOUBERT
GStreamer

- Use FAIL instead of PASS when a test does not succeed.
- The summary must reflect actual test results. A script that prints PASS without verifying the condition will not receive credit.

RECORDING VERIFICATION

- Use an appropriate tool to inspect or decode the recording.
- Possible approaches include:

```
gst-discoverer-1.0 artifacts/test-recording.mkv
```

  or a GStreamer decoding pipeline ending in fakesink.
- A file being non-empty does not, by itself, prove that it is a valid playable recording.

Philippe FOUBERT GStreamer

REPEATABILITY

- The verification must be repeatable.
- Running the following sequence twice must not require manual cleanup:

```
./scripts/clean.sh
./scripts/build.sh
./scripts/verify.sh

./scripts/clean.sh
./scripts/build.sh
./scripts/verify.sh
```

- Generated artifacts must be stored under `artifacts/` or `build/`, not mixed with source files.

Philippe FOUBERT
GStreamer

DETAILS

Philippe FOUBERT
GStreamer

TEAM ORGANIZATION

- Create a file named CONTRIBUTIONS.md.
- It must contain:
  - the names of all team members;
  - a short description of each member’s contributions;
  - the parts developed collaboratively;
  - the tests performed by each team member;
  - the member or members who reviewed each major component.
- A team may divide the initial implementation work, but the completed system must be reviewed collectively.
- For a three-person team, an initial division might be:
  - one member focuses on the native sender;
  - one member focuses on RTP transmission and reception;
  - one member focuses on recording, diagnostics, and verification.
- The Git history and submitted files may be considered when checking whether the contribution report is consistent with the repository.

Philippe FOUBERT
GStreamer

Philippe FOUBERT
GStreamer

USE OF AI TOOLS

PERMITTED USE

- You may use AI tools, including:
  - ChatGPT;
  - Microsoft Copilot;
  - GitHub Copilot;
  - another code-completion or conversational AI tool.
- You may also use official documentation, source code, forums, and other technical resources.

- AI assistance is permitted for:
  - explaining concepts;
  - proposing pipeline structures;
  - generating initial code;
  - reviewing code;
  - interpreting error messages;
  - suggesting diagnostic commands;
  - generating initial tests;
  - improving documentation.

Philippe FOUBERT
GStreamer

YOUR RESPONSIBILITY

- The team remains responsible for everything it submits.
- Generated code must be:
  - reviewed;
  - understood;
  - tested;
  - corrected where necessary;
  - compatible with the supplied environment.
- The fact that an AI tool produced a command, test, or code fragment is not evidence that it is correct.
- In particular, the verification script must test the real implementation. It must not simulate success or hide failures.

Philippe FOUBERT
GStreamer

REQUIRED DISCLOSURE

- Create a file named AI_USAGE.md.
- It must contain:
    1. the AI tools used;
    2. the team member or members who used each tool;
    3. the purpose for which each tool was used;
    4. at least two representative prompts;
    5. important suggestions retained by the team;
    6. important suggestions rejected or corrected;
    7. at least one limitation or technical error found in an AI response;
    8. the tests used to validate the final solution;
    9. a short explanation of what the team learned.
- If the team did not use an AI tool, state this explicitly.
- A statement such as “We used AI to help with the assignment” is not sufficiently detailed.

Philippe FOUBERT
GStreamer

ASSESSMENT OF AI-ASSISTED WORK

- You will not gain or lose marks simply because you used AI.
- Marks are based on:
  - correctness;
  - validation;
  - reproducibility;
  - technical explanations;
  - diagnostic evidence;
  - consistency between the implementation and documentation.

Philippe FOUBERT GStreamer

REQUIRED REPOSITORY STRUCTURE

USE THE FOLLOWING STRUCTURE

```
student-gstreamer-evaluation/
  ├── README.md
  ├── architecture.md
  ├── AI_USAGE.md
  ├── CONTRIBUTIONS.md
  ├── debug-report.md
  ├── experiment.md
  ├── CMakeLists.txt
  ├── config/
  │   └── team.conf
  ├── src/
  │   └── monitor_sender.[c|cpp]
  ├── scripts/
  │   ├── build.sh
  │   ├── clean.sh
  │   ├── sender.sh
  │   ├── receiver.sh
  │   ├── play-recording.sh
  │   ├── run-experiment.sh
  │   └── verify.sh
  ├── evidence/
  │   ├── pipeline.png
  │   └── measurements.csv
  └── recordings/
      └── sample.mkv
```

- The build/ and artifacts/ directories are generated during evaluation and do not have to be committed.
- Do not submit:
  - a complete virtual machine;
  - the complete Codespace;
  - build caches;
  - large core dumps;
  - large raw debug logs;
  - dependencies already provided by the environment;
  - credentials, tokens, or private keys.

Philippe FOUBERT GStreamer

README REQUIREMENTS

- The first section of README.md must be titled:

```
## Quick evaluation
```

- It must contain the exact commands needed to evaluate the submission.

  For example:

```
git clone <repository>
cd <repository>
./scripts/clean.sh
./scripts/build.sh
./scripts/verify.sh
```

- The teacher must not have to search through the document to determine how to run the project.

- The README must also document:

  - required dependencies;
  - expected build time;
  - expected verification time;
  - assigned team configuration;
  - location of generated artifacts;
  - how to launch the graphical preview;
  - how to launch the graphical network receiver;
  - how to play the recording;
  - known limitations.

Philippe FOUBERT GStreamer

ASSESSMENT

The assignment is marked out of 20.

Criterion	Points
Pipeline architecture	3.0
Network transmission and receiver	2.0
Recording and playback	2.0
Native application	4.0
Debugging report	2.0
Engineering experiment	2.0
Automated verification and reproducibility	3.0
Documentation and teamwork	1.5
AI disclosure and validation	0.5
Total	**20.0**

Philippe FOUBERT GStreamer

PIPELINE ARCHITECTURE

- 3 points
- Assessment includes:
  - correct source configuration;
  - correct caps;
  - correct overlays;
  - appropriate use of tee;
  - independent branches;
  - appropriate placement of queue;
  - clear architecture documentation.

Philippe FOUBERT GStreamer

NETWORK TRANSMISSION AND RECEIVER

- 2 points
- Assessment includes:
  - suitable encoder;
  - correct RTP payloader;
  - correct UDP transmission;
  - correct receiver caps;
  - correct depayloader and decoder;
  - functional display and test modes.

Philippe FOUBERT GStreamer

RECORDING AND PLAYBACK

- 2 points
- Assessment includes:
  - suitable encoder and container;
  - correct muxing;
  - clean EOS handling;
  - valid, playable output;
  - reproducible playback command.

Philippe FOUBERT GStreamer

NATIVE APPLICATION

- 4 points
- Assessment includes:
  - command-line processing;
  - pipeline creation;
  - state management;
  - GLib main loop;
  - bus-message processing;
  - meaningful error reporting;
  - EOS handling;
  - timed termination;
  - headless mode;
  - dynamic property changes;
  - resource cleanup.

Philippe FOUBERT GStreamer

DEBUGGING REPORT

- 2 points
- Assessment includes:
  - systematic investigation;
  - appropriate diagnostic tools;
  - relevant evidence;
  - correct root cause;
  - technically accurate correction;
  - reproducibility of the reported issue.

Philippe FOUBERT GStreamer

ENGINEERING EXPERIMENT

- 2 points
- Assessment includes:
  - meaningful comparison;
  - controlled variables;
  - reproducible measurements;
  - appropriate discussion of limitations;
  - evidence-based conclusion.

Philippe FOUBERT GStreamer

AUTOMATED VERIFICATION AND REPRODUCIBILITY

- 3 points
- Assessment includes:
  - clean automated build;
  - useful dependency checks;
  - finite execution time;
  - headless sender and receiver testing;
  - actual verification of RTP reception;
  - actual validation of the recording;
  - proper process cleanup;
  - accurate machine-readable summary;
  - useful logs;
  - repeatable execution.
- The verification script is part of the assessed software, not only a convenience for the teacher.

Philippe FOUBERT
GStreamer

DOCUMENTATION AND TEAMWORK

- 1.5 points
- Assessment includes:
  - clear quick-evaluation instructions;
  - informative architecture diagram;
  - readable repository organization;
  - consistency between documentation and implementation;
  - concrete contribution records;
  - evidence of collective review.

Philippe FOUBERT GStreamer

AI DISCLOSURE AND VALIDATION

- 0.5 points
- Assessment includes:
  - concrete disclosure;
  - representative prompts;
  - critical evaluation of AI output;
  - documented validation.

Philippe FOUBERT
GStreamer

ESSENTIAL REQUIREMENTS

The final mark cannot exceed 9.5/20 if any of the following applies:

- the project cannot be built using the documented procedure;
- the sender cannot be started;
- neither RTP reception nor valid recording can be demonstrated;
- normal termination does not finalize the recording;
- the submitted verification procedure reports success without testing the corresponding behavior.

A compilation failure does not automatically result in zero. Documentation, architecture, experimentation, and debugging evidence may still receive partial credit, but functional criteria cannot receive full credit.

Philippe FOUBERT
GStreamer

SUBMISSION CHECKLIST

Before submitting, verify that:

- [ ] The names of all team members appear in README.md.
- [ ] The assigned configuration appears in config/team.conf.
- [ ] The project builds from a clean directory.
- [ ] monitor_sender –help works.
- [ ] Invalid arguments return a non-zero exit status.
- [ ] The local preview works.
- [ ] Headless mode works.
- [ ] The network receiver works in display mode.
- [ ] The network receiver works in test mode.
- [ ] RTP data is actually received.
- [ ] The recording is finalized using EOS.
- [ ] The recording is playable or decodable.
- [ ] Timed termination works.
- [ ] The dynamic property changes while the pipeline is running.

Philippe FOUBERT GStreamer

- [ ] Dynamic changes are logged in headless mode.
- [ ] architecture.md contains a complete diagram.
- [ ] debug-report.md contains selected diagnostic evidence.
- [ ] experiment.md documents a controlled comparison.
- [ ] evidence/measurements.csv contains measured data.
- [ ] evidence/pipeline.png comes from the implemented pipeline.
- [ ] CONTRIBUTIONS.md describes each member’s work.
- [ ] AI_USAGE.md accurately describes the team’s use of AI.
- [ ] clean.sh, build.sh, and verify.sh are executable.
- [ ] verify.sh has a finite execution time.
- [ ] verify.sh stops all processes that it starts.
- [ ] verify.sh produces an accurate summary.
- [ ] Running the full verification sequence twice succeeds.
- [ ] Generated files are stored under build/ or artifacts/.
- [ ] The repository contains no credentials, tokens, or private keys.

Philippe FOUBERT
GStreamer